

Shell Scripting

By Varun Kumar

Agenda

- Overview
- Advantages of shell script
- Shell variables
- Shell loops
- Debugging Shell Scripts
- **Text Processing and Manipulation commands**
- **Compression and Archiving commands**
- **Scripting and Automation commands**
- **Example of shell scripting**

Overview

- Shell scripting is a way to automate tasks in a Linux environment by writing a series of commands in a text file, which is then executed by a shell interpreter. A shell is a command-line interface that allows users to interact with the operating system. Common shell interpreters start with `#!/bin/bash`.

Why Use Shell Scripting?

- **Automation:** Reduces manual effort for routine tasks.
- **Batch Processing:** Executes multiple commands at once.
- **System Administration:** Used for backups, monitoring, and user management.
- **Customization:** Tailor the system's behavior to suit specific needs.
- **Efficiency:** Saves time by reducing repetitive tasks.

Advantages of shell script

- The command and syntax are exactly the same as those directly entered in command line, so programmer do not need to switch to entirely different syntax.
- Writing shell script are much quicker.
- Quick start
- Interactive debugging etc.

Shell Variables

- Variables in shell scripts store data values that can be reused throughout the script. They help in making scripts dynamic and flexible.

a. Declaring Variables

Assign values using = (no spaces around =).

```
NAME="John"
```

```
echo "Hello, $NAME"
```

Output

```
Hello, John
```

b. Command-line Arguments

```
#!/bin/bash
```

```
echo "First argument: $1"
```

```
echo "Second argument: $2"
```

c. User Input (read Command)

```
echo "Enter your name:"
```

```
read user_name
```

```
echo "Hello, $user_name!"
```

d. export to Make Variables Global

A variable defined inside a script is local by default. Use `export` to make it available system-wide.

```
export MY_VAR="GlobalVariable"  
echo $MY_VAR
```


Shell Loop

- A loop is a powerful programming tool that enables you to execute a set of commands repeatedly.

Types

1. For loop

```
#!/bin/bash
```

```
for fruit in Apple Mango Banana
```

```
do
```

```
    echo "I like $fruit"
```

```
Done
```

Using a Range

```
#!/bin/bash
```

```
for i in {1..5}
```

```
do
```

```
    echo "Number: $i"
```

```
done
```

2. While Loop

Repeats a block of code as long as a condition is true.

```
#!/bin/bash
```

```
count=1
```

```
while [ $count -le 5 ]
```

```
do
```

```
    echo "Count: $count"
```

```
    ((count++))
```

```
done
```

3. Nested Loops

Loops inside another loop for multi-level iterations.

```
#!/bin/bash

for i in {1..3}
do
    for j in {1..3}
    do
        echo -n "$((i * j)) "
    done
    echo
done
```


Debugging Shell Scripts

1. Using `set -x` for Debugging

- The `set -x` command enables debugging mode, which prints each command and its arguments as they are executed.
- This helps in understanding how the script is being executed step by step.

```
#!/bin/bash

set -x # Enable debugging | set -e #exit | set -u unset variables | set -v verbose mode | set -o pipeline (Ensures that the whole pipeline fails if any command in it fails)

echo "Starting the script"

VAR="Hello"

echo $VAR

set +x # Disable debugging

echo "Script execution completed"
```

2. Using trap to Capture Signals

- The `trap` command is used to capture and handle signals like `SIGINT` (Ctrl+C) and `SIGTERM`.
- This ensures proper cleanup before the script exits unexpectedly.

```
#!/bin/bash
```

```
trap "echo 'Script interrupted! Cleaning up...'; exit 1" SIGINT SIGTERM
```

```
echo "Press Ctrl+C to test"
```

```
while true; do
```

```
    sleep 1
```

```
done
```


Text Processing and Manipulation

- `grep` - Search text using patterns
- `sed` - Stream editor used to **search, find, replace, delete, and modify text** (`sed 's/apple/banana/g' fruits.txt`) (`sed -n '3p' file.txt`) (`sed '5d' file.txt`) (`sed '/^$/d' file.txt`) to delete empty lines in script file
- `awk` - Pattern scanning and processing language (`awk '{ print $1, $3 }' file.txt`) (`awk '/error/ { print $0 }' logfile.txt`) (`awk -F ':' '{ print $1, $3 }' /etc/passwd`)
- `cut` - Remove sections from each line of files (`cut -d ',' -f1 data.csv`)
- `sort` - Sort lines of text files (`sort -n numbers.txt`) (`sort -r names.txt`) (`sort -k2 -n data.txt`)
- `uniq` - Report or Remove or omit repeated lines (`uniq -d names.txt`) (`uniq names.txt`)
- `tr` - Translate or delete characters (`echo "hello world" | tr 'a-z' 'A-Z'`) (`echo "hello 123" | tr -d '0-9'`)

Compression and Archiving

- tar - Archiving utility (tar -cvf TDP.tar.gz TDP)
- gzip - Compress or expand files (gzip myfile.txt) unzip (gunzip myfile.txt.gz)
- zip / unzip - Package and compress/decompress files (zip -r my_archive.zip my_folder/) (unzip my_archive.zip)
- rsync - Remote file synchronization (rsync -avz /home/varun/docs/user@192.168.0.56:/home/user/backup/)

Incremental Transfer

Speed & Efficiency

Compression Support

Tool	Archives	Compresses	Works on Folders
tar	✓ Yes	✗ No	✓ Yes
gzip	✗ No	✓ Yes	✗ No (file only)
tar + gzip	✓ Yes	✓ Yes	✓ Yes
zip	✓ Yes	✓ Yes	✓ Yes

Scripting and Automation

- echo - Display a line of text
- read - Read from standard input
- export - Set environment variables
- source / . - Execute commands from a file
- crontab - Schedule periodic jobs
- find - Search for files in a directory hierarchy
- xargs - Build and execute command lines from standard input

Example of shell scripting

1. For our first shell script, we will write a script which say "Hello World"

Input

```
#!/bin/bash
```

```
#This is a comment!
```

```
echo "Hello World"
```

Output

```
Hello World
```

Thank you

Linux[™]
SHELL
SCRIPTING[™]

